

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7

по курсу Объектно-ориентированное программирование

Выполнил: Акст Роман, 6203-010302D

Задание 1

Интерфейс `TabulatedFunction` был расширен параметризованным типом `Iterable<FunctionPoint>`, что потребовало реализации метода `iterator()` в классах `ArrayTabulatedFunction` и `LinkedListTabulatedFunction`. Итераторы были реализованы как анонимные классы, эффективно работающие с внутренней структурой объектов, без использования публичных методов табулированных функций. Метод `remove()` в итераторах выбрасывает `UnsupportedOperationException`, а метод `next()` генерирует `NoSuchElementException` при отсутствии следующего элемента. На рисунке 1 – реализация итератора `ArrayTabulatedFunction`, на рисунке 2 – реализация итератора `LinkedListTabulatedFunction`, на рисунке 3 – результат работы кода проверки из задания.

```
@Override new *
public Iterator<FunctionPoint> iterator()
{
    return new Iterator<FunctionPoint>() new *
    {
        private int index; 2 usages

        @Override new *
        public boolean hasNext()
        {
            return index < amountOfElements;
        }

        @Override new *
        public FunctionPoint next()
        {
            if(!hasNext())
                throw new NoSuchElementException("Невозможно получить следующий элемент");

            return points[index++];
        }

        @Override new *
        public void remove()
        {
            throw new UnsupportedOperationException("Невозможно выполнить remove()");
        }
    };
}
```

Рисунок 1

```

@Override new *
public Iterator<FunctionPoint> iterator()
{
    return new Iterator<FunctionPoint>() new *
    {
        private FunctionNode currentNode = head.next; 4 usages

        @Override new *
        public boolean hasNext()
        {
            return currentNode != head;
        }

        @Override new *
        public FunctionPoint next()
        {
            if(!hasNext())
                throw new NoSuchElementException("Невозможно получить следующий элемент");

            FunctionNode returnableNode = currentNode;
            currentNode = currentNode.next;

            return returnableNode.point;
        }

        @Override new *
        public void remove()
        {
            throw new UnsupportedOperationException("Невозможно выполнить remove()");
        }
    };
}

```

Рисунок 2

First task:

```

(1.0;0.0)
(2.0;0.0)
(3.0;0.0)
(4.0;0.0)
(5.0;0.0)
(6.0;0.0)
(7.0;0.0)
(8.0;0.0)
(9.0;0.0)
(10.0;0.0)

```

Рисунок 3

Задание 2

Был применен паттерн "Фабричный метод" для динамического выбора типа создаваемых объектов табулированных функций. Был описан интерфейс `TabulatedFunctionFactory` с тремя перегруженными методами `createTabulatedFunction()`, соответствующими конструкторам классов табулированных функций. В классах `ArrayTabulatedFunction` и `LinkedListTabulatedFunction` были добавлены публичные вложенные классы фабрик, реализующие этот интерфейс. В классе `TabulatedFunctions` было объявлено статическое поле для хранения текущей фабрики, а также метод `setTabulatedFunctionFactory()` для ее замены. Существующие методы создания объектов в `TabulatedFunctions` были модифицированы для использования фабрики вместо прямого вызова конструкторов. На рисунке 4 – реализация методов `createTabulatedFunction`, рисунки 5-6 – вложенные классы фабрик в `ArrayTabulatedFunction` и `LinkedListTabulatedFunction` соответственно. На рисунке 7 - результат работы кода проверки из задания.

```
private static TabulatedFunctionFactory tabulatedFunctionFactory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory(); 7 usages

public static void setTabulatedFunctionFactory(TabulatedFunctionFactory tff) 2 usages new *
{
    tabulatedFunctionFactory = tff;
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) no usages new *
    throws IllegalArgumentException
{
    return tabulatedFunctionFactory.createTabulatedFunction(leftX, rightX, pointsCount);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) no usages new *
    throws IllegalArgumentException
{
    return tabulatedFunctionFactory.createTabulatedFunction(leftX, rightX, values);
}

public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) no usages new *
    throws IllegalArgumentException
{
    return tabulatedFunctionFactory.createTabulatedFunction(points);
}
```

Рисунок 4

```
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory 2 usages new *
{
    @Override 2 usages new *
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointCount) throws IllegalArgumentException {
        return new ArrayTabulatedFunction(leftX, rightX, pointCount);
    }

    @Override 1 usage new *
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) throws IllegalArgumentException {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override 3 usages new *
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) throws IllegalArgumentException {
        return new ArrayTabulatedFunction(points);
    }
}
```

Рисунок 5

```

public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory 1 usage new *
{
    @Override 2 usages new *
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointCount) throws IllegalArgumentException {
        return new LinkedListTabulatedFunction(leftX, rightX, pointCount);
    }

    @Override 1 usage new *
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) throws IllegalArgumentException {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    @Override 3 usages new *
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) throws IllegalArgumentException {
        return new LinkedListTabulatedFunction(points);
    }
}

```

Рисунок 6

Second task:

```
class functions.ArrayTabulatedFunction
```

```
class functions.LinkedListTabulatedFunction
```

```
class functions.ArrayTabulatedFunction
```

Рисунок 7

Задание 3

В классе `TabulatedFunctions` были добавлены три перегруженных метода `createTabulatedFunction()`, принимающие в качестве параметра ссылку типа `Class` на класс, реализующий интерфейс `TabulatedFunction`. Эти методы находят в указанном классе конструктор с соответствующими параметрами и используют его для создания объекта. В случае возникновения исключений при рефлексивных операциях (например, `NoSuchMethodException`) они преобразуются в `IllegalArgumentException` с сохранением причины. На рисунке 8 – пример перегрузки метода `createTabulatedFunction`, на рисунке 9 – пример перегрузки `inputTabulatedFunction`, на рисунке 10 - результат работы кода проверки из задания.

```
public static TabulatedFunction createTabulatedFunction(Class<?> functionClass, FunctionPoint[] points) 1 usage new *
    throws IllegalArgumentException
{
    if(TabulatedFunction.class.isAssignableFrom(functionClass))
    {
        try
        {
            java.lang.reflect.Constructor<?> constructor = functionClass.getDeclaredConstructor(
                FunctionPoint[].class
            );

            return (TabulatedFunction) constructor.newInstance((Object) points);
        }
        catch (NoSuchMethodException e)
        {
            throw new IllegalArgumentException("Класс не имеет конструктора");
        }
        catch (Exception e)
        {
            throw new RuntimeException("Возникла ошибка при создании табулированной функции");
        }
    }
    else
    {
        throw new IllegalArgumentException("Был передан недопустимый класс");
    }
}
```

Рисунок 8

```

public static TabulatedFunction inputTabulatedFunction(Class<?> functionClass, InputStream in)  no usages  new *
{
    if(TabulatedFunction.class.isAssignableFrom(functionClass))
    {
        try
        {
            DataInputStream inputStream = new DataInputStream(in);
            int pointsCount = inputStream.readInt();

            FunctionPoint[] points = new FunctionPoint[pointsCount];

            java.lang.reflect.Constructor<?> constructor = functionClass.getDeclaredConstructor(
                FunctionPoint[].class
            );

            for(int i = 0; i < pointsCount; i++)
            {
                double x = inputStream.readDouble();
                double y = inputStream.readDouble();

                points[i] = new FunctionPoint(x, y);
            }

            return (TabulatedFunction) constructor.newInstance((Object) points);
        }
        catch (NoSuchMethodException e)
        {
            throw new IllegalArgumentException("Класс не имеет конструктора");
        }
        catch (IOException e)
        {
            throw new RuntimeException(e);
        }
        catch (Exception e)
        {
            throw new RuntimeException("Возникла ошибка при создании табулированной функции");
        }
    }
    else
    {
        throw new IllegalArgumentException("Был передан недопустимый класс");
    }
}

```

Рисунок 9

```

Third task:
class functions.ArrayTabulatedFunction
{(0,00; 0,00), (5,00; 0,00), (10,00; 0,00)}
class functions.ArrayTabulatedFunction
{(0,00; 0,00), (10,00; 10,00)}
class functions.LinkedListTabulatedFunction
{(0,00; 0,00), (10,00; 10,00)}
class functions.LinkedListTabulatedFunction
{(0,00; 0,00), (0,31; 0,31), (0,63; 0,59), (0,94; 0,81), (1,26; 0,95), (1,57; 1,00), (1,88; 0,95), (2,20; 0,81), (2,51; 0,59), (2,83; 0,31), (3,14; 0,00)}

```

Рисунок 10